

ReactJS Full Course

Complete Study Notes: From Basics to Advanced

Based on the topics covered in the MERN Stack / React Full Course video by
Sheryians Coding School (Instructor: Sarthak Sharma)

Includes: Core Concepts • Hooks • Routing • Context API • Real Project
Patterns • React Interview Q&A

Table of Contents

1. Introduction to React & React Overview
2. Import & Export in JavaScript/React
3. Real DOM vs Virtual DOM
4. Setting Up React with Vite
5. Components Explained
6. Understanding Props
7. Mini Projects Using Props
8. Styling with CSS & Tailwind CSS
9. Working with Functions in React
10. Understanding Hooks (useState, useEffect, etc.)
11. Advanced State Management
12. Form Handling in React
13. Two-Way Data Binding
14. Notes App Project (Concept Walkthrough)
15. Using LocalStorage
16. API Calls in React
17. useEffect Hook Explained (Deep Dive)
18. Gallery Project (Concept Walkthrough)
19. React Router DOM Basics
20. Advanced Routing
21. Context API (Introduction, Children as Props, In Action)
22. Bonus Concepts & Best Practices
23. React Interview Questions & Answers

1. Introduction to React & React Overview

React is a free, open-source JavaScript library (created and maintained by Meta) used for building user interfaces, especially single-page applications where data changes over time without full page reloads. React lets you build encapsulated components that manage their own state, then compose them to make complex UIs.

Why React?

- Component-based architecture makes UI code reusable and easier to maintain.
- Uses a Virtual DOM for fast, efficient updates to the actual webpage.
- Declarative style: you describe what the UI should look like for a given state, and React handles the how.
- Huge ecosystem and community support (routing, state management, UI libraries, etc.).
- Backed by Meta and used by companies like Facebook, Instagram, Netflix, and Airbnb.

Core Idea

React applications are built as a tree of components. Each component receives inputs (props) and internal data (state) and returns JSX describing what should appear on the screen. When state or props change, React efficiently re-renders only the parts of the UI that need to change.

2. Import & Export in JavaScript/React

Modern JavaScript (ES Modules) allows splitting code into multiple files/modules and sharing functionality between them using import and export statements. This is essential in React because every component typically lives in its own file.

Named Export/Import

```
export const add = (a, b) => a + b;  
import { add } from './mathUtils';
```

Default Export/Import

```
export default function App() { ... }  
import App from './App';
```

A file can have multiple named exports but only one default export. Named imports must match the exported name exactly (or use aliasing with 'as'), while default imports can be given any name.

3. Real DOM vs Virtual DOM

Real DOM	Virtual DOM
Direct representation of the actual webpage structure.	A lightweight JavaScript object copy of the Real DOM.
Updating it is slow and expensive because the whole DOM tree must be rebuilt.	Updates are fast as only the changed parts are recalculated and re-rendered.
Every change directly manipulates the whole DOM tree.	React compares (diffs) the new Virtual DOM with the previous one and updates only the differences.
Can cause performance issues in large, dynamic pages.	Improves performance through efficient reconciliation.

React's reconciliation algorithm compares the previous and current Virtual DOM trees using a diffing algorithm, then applies only the minimal set of changes needed to the Real DOM — this is why React apps feel fast even with frequent UI updates.

4. Setting Up React with Vite

Vite is a modern, fast build tool for frontend projects. It's commonly preferred over Create React App because of its near-instant dev server startup and lightning-fast hot module replacement (HMR).

Basic Setup Commands

```
npm create vite@latest my-app -- --template react
cd my-app
npm install
npm run dev
```

Typical Project Structure

- index.html – entry HTML file with a root div.
- src/main.jsx – entry point that renders the root React component.
- src/App.jsx – the main App component.
- src/components/ – folder for reusable components.
- vite.config.js – Vite configuration file.

5. Components Explained

Components are the building blocks of a React app — independent, reusable pieces of UI. Modern React primarily uses functional components (plain JavaScript functions that return JSX).

```
function Greeting() {
  return <h1>Hello, World!</h1>;
}
export default Greeting;
```

Types of Components

- Functional Components – simple JS functions returning JSX; the modern standard, used with Hooks.
- Class Components – ES6 classes extending `React.Component`; the older pattern, less common now.
- Presentational vs Container Components – a design pattern separating UI rendering from logic/data-fetching.

6. Understanding Props

Props (short for 'properties') are read-only inputs passed from a parent component to a child component. They let you make components dynamic and reusable.

```
function Welcome(props) {
  return <h2>Welcome, {props.name}!</h2>;
}
// Usage: <Welcome name="Kunal" />
```

Props flow one-way (parent → child) which is called 'unidirectional data flow'. This makes apps more predictable and easier to debug.

7. Mini Projects Using Props

Small hands-on projects (like profile cards, product listing cards, or a simple greeting app) are a great way to practice passing and destructuring props, rendering lists with `.map()`, and using the special 'key' prop for list items.

```
const users = ['Amit', 'Riya', 'Sara'];
<ul>
  {users.map((user, i) => <li key={i}>{user}</li>)}
</ul>
```

8. Styling with CSS & Tailwind CSS

Ways to Style React Components

- Plain CSS files imported into components (import './App.css').
- CSS Modules for scoped, component-level styling (Button.module.css).
- Inline styles using JS objects: `<div style={{ color: 'red' }}>`.
- CSS-in-JS libraries like styled-components.
- Utility-first frameworks like Tailwind CSS.

Tailwind CSS

Tailwind is a utility-first CSS framework where you style elements by composing small, single-purpose classes directly in your markup (e.g., `flex`, `p-4`, `text-center`, `bg-blue-500`) instead of writing custom CSS. It speeds up UI development and keeps styling consistent across a project.

```
<button className="px-4 py-2 bg-blue-600 text-white rounded-lg hover:bg-blue-700">
  Click Me
</button>
```

9. Working with Functions in React

Since React components are just functions, strong fundamentals in JavaScript functions are essential: regular functions, arrow functions, callback functions, and event-handler functions.

```
function handleClick() {
  console.log('Button clicked!');
}
<button onClick={handleClick}>Click</button>
```

Arrow functions are especially common in React for concise event handlers and for passing arguments: `onClick={() => handleDelete(item.id)}`.

10. Understanding Hooks

Hooks are special functions (introduced in React 16.8) that let functional components 'hook into' React features like state and lifecycle, without writing class components. Hook names always start with 'use'.

Commonly Used Hooks

- `useState` – adds local state to a functional component.
- `useEffect` – runs side effects (data fetching, subscriptions, DOM manipulation) after render.
- `useContext` – reads values from React Context without prop drilling.
- `useRef` – persists a mutable value across renders / accesses DOM nodes directly.
- `useMemo` / `useCallback` – memoize expensive calculations or functions to optimize performance.

```
import { useState } from 'react';
function Counter() {
  const [count, setCount] = useState(0);
  return <button onClick={() => setCount(count + 1)}>{count}</button>;
}
```

Rules of Hooks

- Only call Hooks at the top level — never inside loops, conditions, or nested functions.
- Only call Hooks from React function components or custom Hooks.

11. Advanced State Management

As apps grow, managing state across many components becomes harder. Beyond `useState` in a single component, React offers patterns and tools for sharing and organizing state at scale.

- Lifting state up – moving shared state to the closest common parent component.
- `useReducer` – manages more complex state logic using a reducer function, similar to Redux's pattern.
- Context API – shares state globally without passing props through every level ('prop drilling').
- External libraries – Redux Toolkit, Zustand, or Recoil for large-scale, app-wide state management.

12. Form Handling in React

React forms are typically built using 'controlled components', where form input values are driven by React state rather than the DOM itself. This gives full control over validation and submission logic.

```
const [name, setName] = useState('');
<input value={name} onChange={(e) => setName(e.target.value)} />
<button onClick={() => console.log(name)}>Submit</button>
```

13. Two-Way Data Binding

Two-way data binding means the UI updates the state and the state updates the UI, keeping both in sync. In React this is achieved manually (unlike frameworks like Angular/Vue which have built-in directives) by combining the input's 'value' attribute with an 'onChange' handler that updates state.

14. Notes App Project (Concept Walkthrough)

A classic beginner project that ties together components, props, state, forms, and list rendering. Typical features include:

- A text input (controlled component) to add a new note.
- State (array) holding all notes, updated with `setNotes([...notes, newNote])`.
- Rendering the notes list using `.map()` with unique 'key' props.
- Delete functionality using `.filter()` to remove a note by id.
- Optionally persisting notes using `LocalStorage` so they survive a page refresh.

15. Using LocalStorage

`LocalStorage` is a browser API that lets you store key-value data (as strings) that persists even after the page is closed and reopened. It's commonly combined with `useEffect` in React to save and load app state.

```
// Save localStorage.setItem('notes', JSON.stringify(notes));

// Load const saved = JSON.parse(localStorage.getItem('notes')) || [];
```

16. API Calls in React

React apps commonly fetch data from external APIs using the built-in `fetch` API or libraries like `axios`, usually inside a `useEffect` so the call happens after the component mounts.

```
useEffect(() => {
  fetch('https://api.example.com/users')
    .then(res => res.json())
    .then(data => setUsers(data));
}, []);
```

17. useEffect Hook Explained (Deep Dive)

`useEffect` lets you perform side effects — actions outside of pure rendering, like data fetching, timers, subscriptions, or manually changing the DOM. It runs after the render is committed to the screen.

The Dependency Array Controls When It Runs

- `useEffect(() => {...})` — no array: runs after every render.
- `useEffect(() => {...}, [])` — empty array: runs only once, after the first render (like `'componentDidMount'`).
- `useEffect(() => {...}, [count])` — runs whenever `'count'` changes.
- Returning a function from `useEffect` defines cleanup logic, run before the next effect or on unmount (like `'componentWillUnmount'`).

18. Gallery Project (Concept Walkthrough)

A gallery/image-fetching project usually combines several concepts learned so far:

- Fetching image data from a public API (e.g., `Unsplash` or `Pexels`) using `useEffect`.
- Storing the fetched results in state and rendering them in a responsive grid using `CSS/Tailwind`.
- Adding a search bar (controlled input) that re-triggers the API call.

- Handling loading and error states gracefully while data is being fetched.

19. React Router DOM Basics

React itself doesn't include routing — 'react-router-dom' is the standard library for handling navigation between different views/pages in a single-page application without a full page reload.

```
import { BrowserRouter, Routes, Route } from 'react-router-dom';

<BrowserRouter>
  <Routes>
    <Route path="/" element={<Home />} />
    <Route path="/about" element={<About />} />
  </Routes>
</BrowserRouter>
```

20. Advanced Routing

- Dynamic routes with URL parameters — <Route path="/user/:id" /> and useParams() to read them.
- Nested routes — rendering child routes inside a parent layout using <Outlet />.
- Programmatic navigation — using the useNavigate() hook to redirect users after an action (e.g., after login).
- Protected/private routes — restricting access to certain routes based on authentication state.
- The Link and NavLink components for client-side navigation without full page reloads.

21. Context API (Introduction, Children as Props, In Action)

The Context API is React's built-in solution for sharing data (like theme, authenticated user, or language) across many components without manually passing props down through every level ('prop drilling').

Children as Props

'props.children' represents whatever is nested between a component's opening and closing tags. It's what allows building flexible wrapper/layout components.

```
function Card({ children }) {
  return <div className="card">{children}</div>;
}
// Usage: <Card><p>Hello!</p></Card>
```

Context API in Action

```
const ThemeContext = createContext();

function App() {
  return (
    <ThemeContext.Provider value="dark">
      <Toolbar />
    </ThemeContext.Provider>
  );
}
```



```
function Toolbar() {  
  const theme = useContext(ThemeContext);  
  return <div>Current theme: {theme}</div>;  
}
```

22. Bonus Concepts & Best Practices

- Folder structure — organize by feature or type (components/, pages/, hooks/, context/, utils/, assets/).
- Custom Hooks — extract reusable stateful logic into your own 'use...' functions.
- Code splitting & lazy loading — React.lazy() and Suspense to load components only when needed.
- Error boundaries — catch rendering errors in a part of the UI without crashing the whole app.
- Performance optimization — React.memo, useMemo, useCallback to avoid unnecessary re-renders.
- Environment variables — store API keys/config in .env files (Vite uses the VITE_ prefix).
- Deployment — building with 'npm run build' and hosting on platforms like Vercel or Netlify.

23. React Interview Questions & Answers

A curated set of commonly asked React.js interview questions, covering fundamentals through advanced topics — useful for internships and full-stack developer roles.

Q1. What is React and why is it used?

React is a JavaScript library for building fast, interactive user interfaces using a component-based, declarative approach. It's used because it makes UI code reusable, improves performance via the Virtual DOM, and scales well for large single-page applications.

Q2. What is JSX?

JSX (JavaScript XML) is a syntax extension that lets you write HTML-like code inside JavaScript. It gets compiled (via Babel) into `React.createElement()` calls, which produce React elements.

Q3. What is the Virtual DOM and how does it improve performance?

The Virtual DOM is an in-memory, lightweight copy of the Real DOM. When state changes, React creates a new Virtual DOM tree, diffs it against the previous one, and updates only the changed parts of the Real DOM, avoiding costly full-page re-renders.

Q4. What is the difference between props and state?

Props are read-only data passed from a parent to a child component and cannot be modified by the child. State is data owned and managed within a component itself, and can change over time, triggering re-renders.

Q5. What are Hooks? Name a few commonly used ones.

Hooks are functions that let functional components use React features like state and lifecycle without writing a class. Common ones include `useState`, `useEffect`, `useContext`, `useRef`, `useMemo`, and `useCallback`.

Q6. What is the difference between `useState` and `useReducer`?

`useState` is best for simple, independent pieces of state. `useReducer` is better for complex state logic with multiple sub-values or when the next state depends on the previous one, using a reducer function similar to Redux.

Q7. Explain the `useEffect` Hook and its dependency array.

`useEffect` runs side effects after render. An empty dependency array `[]` runs the effect once on mount; a populated array `[value]` reruns the effect whenever 'value' changes; no array at all reruns it after every render. Returning a function from `useEffect` defines cleanup logic.

Q8. What is prop drilling and how can you avoid it?

Prop drilling is passing data through many nested components that don't need it, just to reach a deeply nested child that does. It can be avoided using the Context API or state management libraries like Redux or Zustand.

Q9. What are controlled vs uncontrolled components?

A controlled component's form data is handled by React state (value + onChange). An uncontrolled component manages its own state internally in the DOM, and React reads its value only when needed, typically via a ref.

Q10. What is the significance of the 'key' prop in lists?

Keys help React identify which items in a list have changed, been added, or removed, so it can efficiently update only those items instead of re-rendering the entire list. Keys should be stable and unique, not array indexes when the list can reorder.

Q11. What is the Context API used for?

The Context API lets you share values (like theme, logged-in user, or language) across the component tree without manually passing props at every level, solving the prop drilling problem for global-ish data.

Q12. What is the difference between React Router's Link and an HTML anchor tag?

A regular `<a>` tag causes a full page reload from the server. React Router's `<Link>` performs client-side navigation, updating the URL and rendering the new component without reloading the whole page.

Q13. What are React Fragments and why use them?

Fragments (`<>...</>` or `<React.Fragment>`) let you group multiple elements without adding an extra node to the DOM, which keeps the DOM tree cleaner than wrapping everything in an unnecessary `<div>`.

Q14. What is the difference between useMemo and useCallback?

`useMemo` memoizes the return value of a computation, recalculating it only when its dependencies change. `useCallback` memoizes a function definition itself, useful for preventing unnecessary re-renders of child components that receive that function as a prop.

Q15. What is the component lifecycle, and how do Hooks relate to it?

Class components have lifecycle methods like `componentDidMount`, `componentDidUpdate`, and `componentWillUnmount`. In functional components, `useEffect` (with different dependency arrays and cleanup functions) replicates all three of these behaviors.

Q16. What is React.memo and when should you use it?

`React.memo` is a higher-order component that memoizes a functional component, skipping re-render if its props haven't changed. It's useful for optimizing components that render often with the same props.

Q17. What is the difference between state and refs (useRef)?

Updating state triggers a re-render of the component; updating a ref (via `useRef`) does not. Refs are used to directly access DOM elements or to store mutable values that shouldn't cause re-renders, like timers or previous values.

Q18. How do you handle forms with multiple inputs in React?

Typically by keeping a single state object with a key for each field, then using one `onChange` handler that updates the correct field dynamically using the input's 'name' attribute: `setForm({ ...form, [e.target.name]: e.target.value })`.

Q19. What is lifting state up?

It's the pattern of moving shared state to the closest common ancestor of the components that need it, so that data and update functions can be passed down as props to keep sibling components in sync.

Q20. What is the difference between synchronous and asynchronous state updates in React?

React state updates via `setState`/`useState` setters are often batched and applied asynchronously for performance, meaning the updated value isn't immediately available right after calling the setter — it's reflected on the next render.

Note: This guide summarizes the topic areas covered in the referenced YouTube video's chapter list, written as original study notes, along with a standard set of commonly asked React interview questions. For hands-on code walkthroughs matching the video exactly, refer to the full video and its source code.